



RxJS В Angular

От фундаментальных принципов
к гибридной реактивной архитектуре.

RxJS — это математическое открытие, а не случайная библиотека.

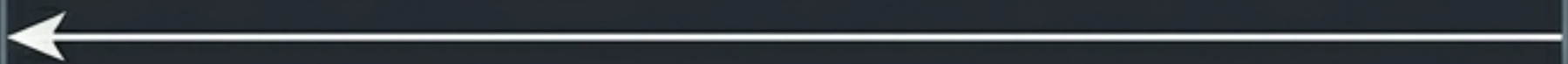
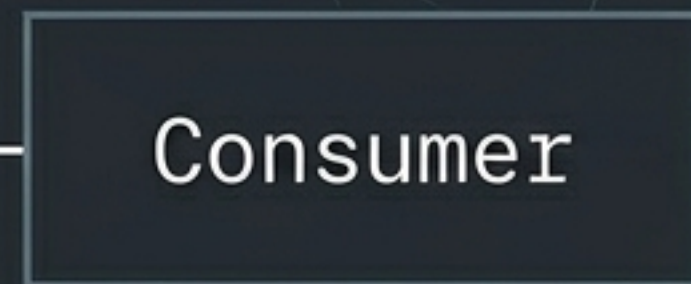
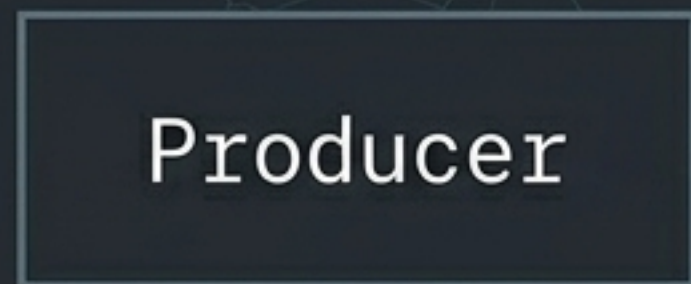
В 2007 году Эрик Майер (Erik Meijer) и команда Microsoft в рамках проекта Volta выявили математическую двойственность. Инвертировав стрелки паттерна Iterator, они создали идеальный Push-based интерфейс для потоков данных.

	Single (Одно значение)	Multiple (Множество значений)
Pull (Синхронный запрос)	Function	Iterator (IEnumerable<T>)
Push (Асинхронная доставка)	Promise	Observable (IObservable<T>)

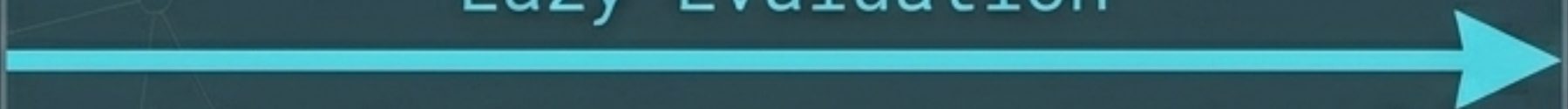
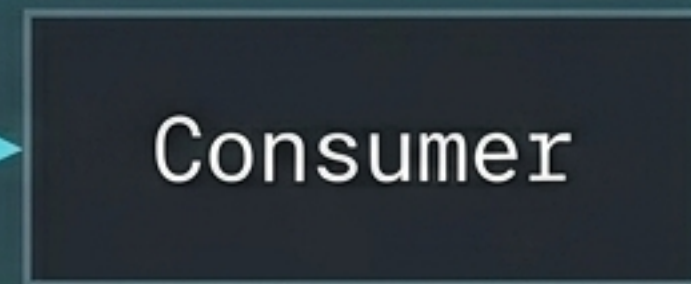
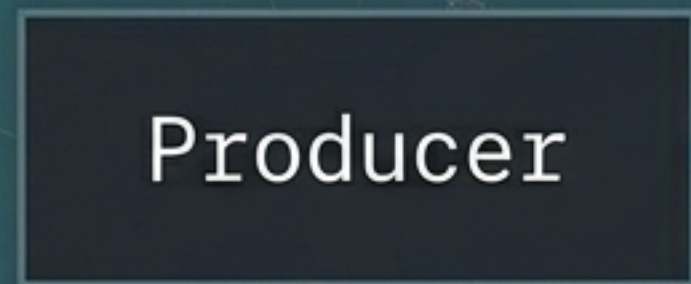
Смена парадигмы: от запроса данных к реакции на них.

Мы переходим от системы, где потребитель контролирует получение данных, к архитектуре, где мы декларативно описываем реакцию на будущие события. Observable — это лениво вычисляемая функция, генерирующая данные.

Pull



Push-based



Lazy Evaluation

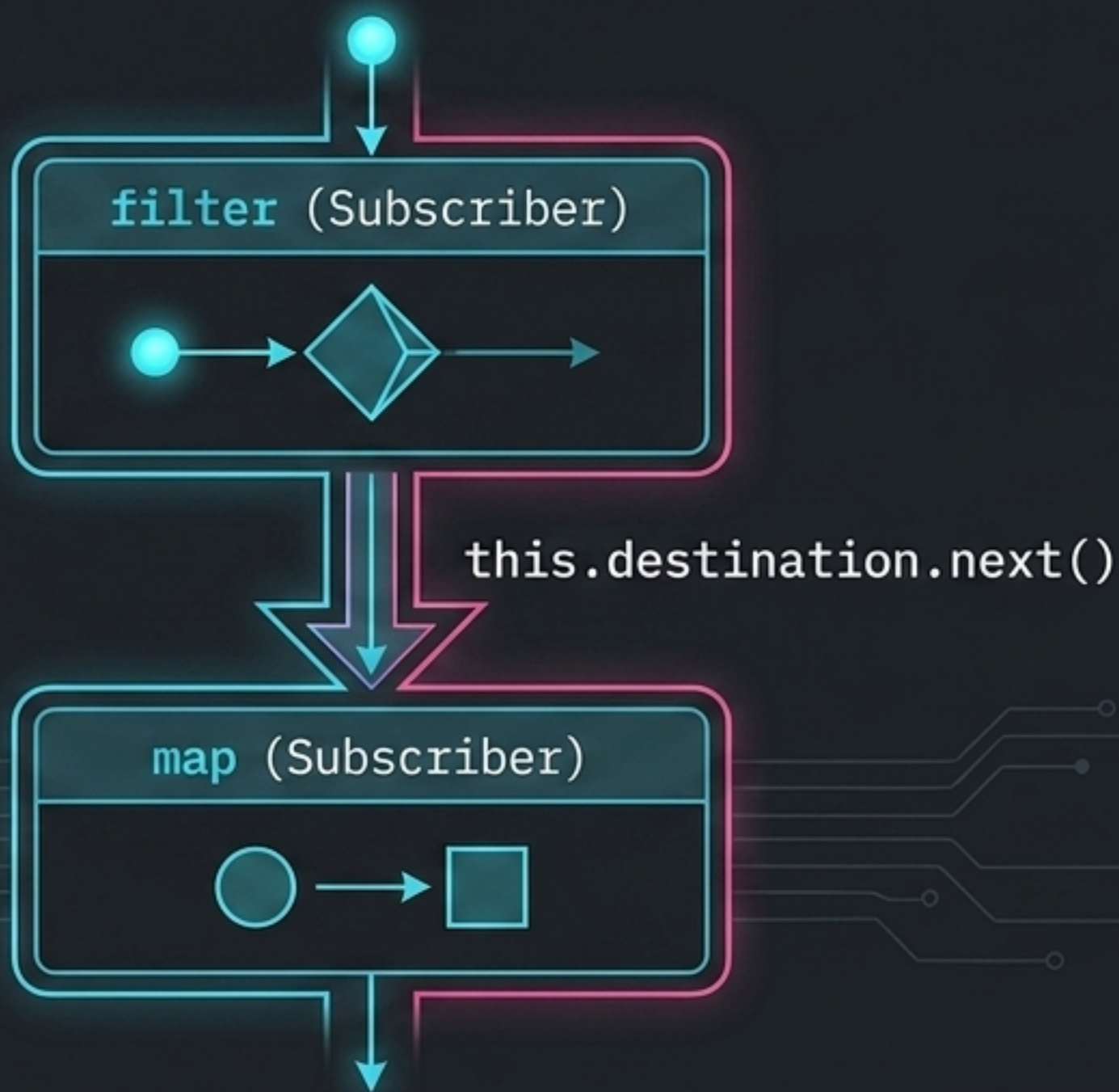
(Ничего не происходит до вызова `.subscribe()`)

Технический анализ: Observable против Promise.

Feature	Promise	Observable
Вычисление	Eager (Выполняется немедленно)	Lazy (Ожидает подписки)
Значения	Single (Только одно)	Multiple (От нуля до бесконечности)
Отмена	Uncancellable (Нельзя отменить)	Cancellable (Полный контроль через <code>unsubscribe()</code>)
Трансляция	Multicast (Разделяет один результат)	Unicast (Новое выполнение для каждого подписчика)

Внутренняя механика: Цепочка Subscriber'ов.

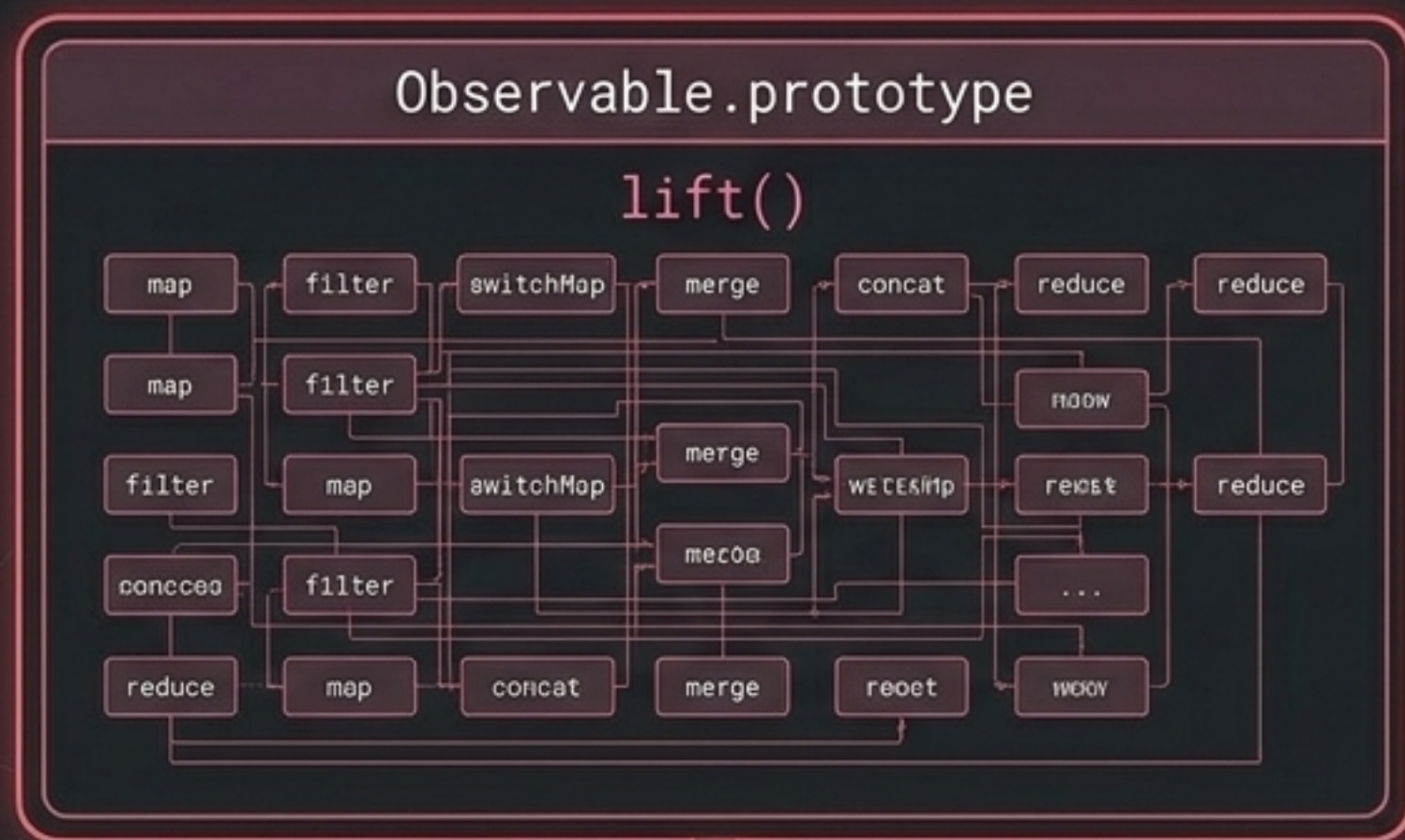
Как работает `pipe()`? Это не магия. Каждое звено в цепи — это объект `Subscriber`, который обрабатывает значение и рекурсивно вызывает метод `_next()` у следующего звена через `this.destination.next()`.



Эволюция архитектуры для современных бандлеров.

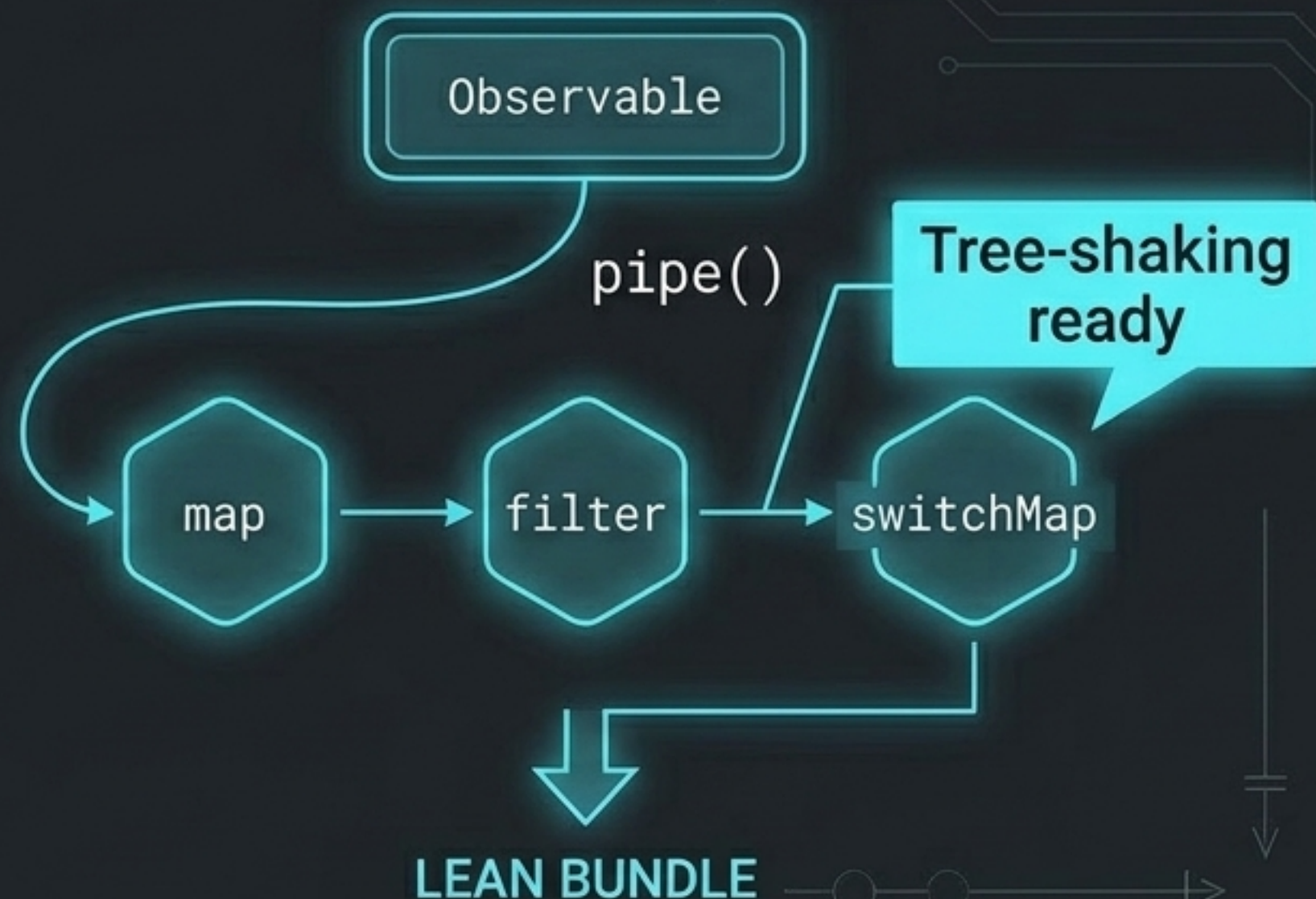
Переход от методов прототипа (`lift()`) к **Pipeable operators** (чистым функциям) позволил современным инструментам сборки применять Tree-shaking, удаляя неиспользуемый код и радикально уменьшая размер бандла.

RxJS < 5.5



BLOATED BUNDLE

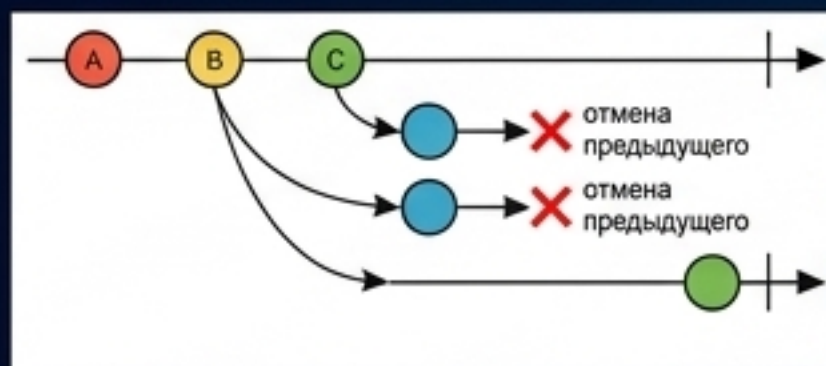
RxJS 5.5+



Управление потоком времени с помощью Flattening Operators: Визуальные примеры и применение

Взят из Slide C, слайда. Управление потоком времени с помощью Flattening Operators: Визуальные примеры и применение операторов. Визуальные операторы позволяют перхватывать, продвигать и изменять диаграммы и визуальный язык, детали диаграмм; Flattening operators: с одним оператором, лучшая читаемость вложенном маркере диаграмм.

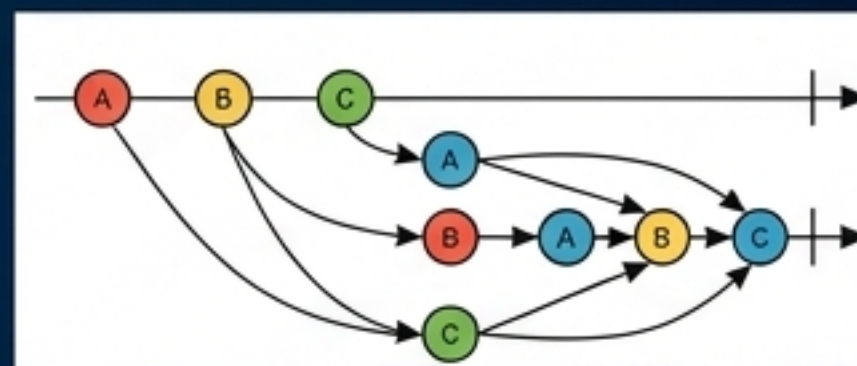
switchMap



Описание:
Отменяет предыдущую подписку на внутренний поток при появлении нового значения.

Применение: Автодополнение поиска, живой поиск.

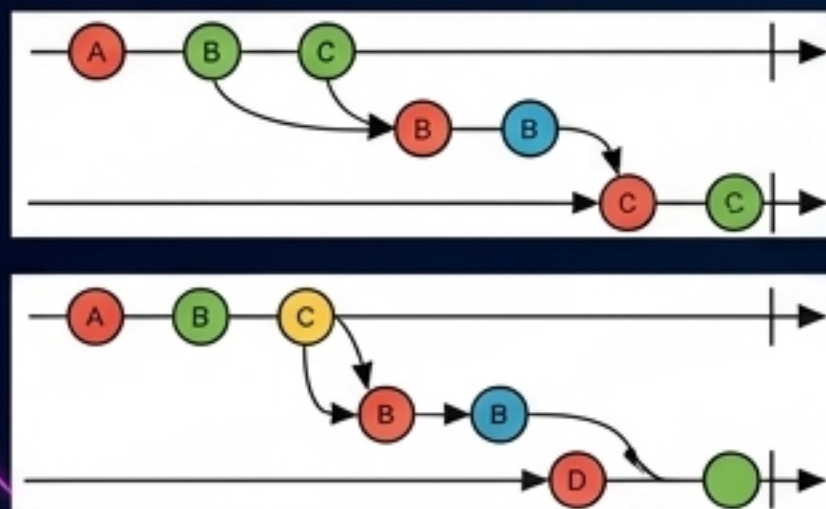
mergeMap



Описание:
Подписывается на все внутренние потоки параллельно, объединяя их результаты по мере поступления.

Применение: Параллельные HTTP-запросы, загрузка множества файлов.

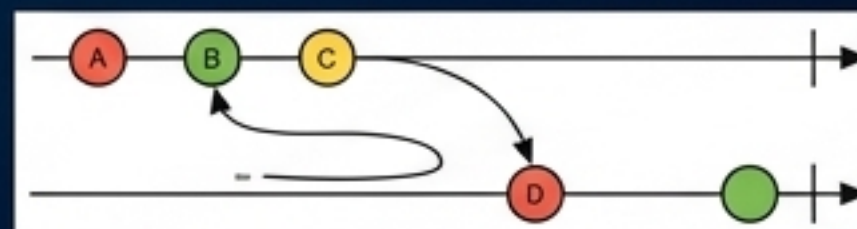
concatMap



Описание: Подписывается на внутренние потоки строго последовательно, ожидая завершения предыдущего.

Применение: Последовательные операции (сохранение, затем чтение), очередь задач.

exhaustMap



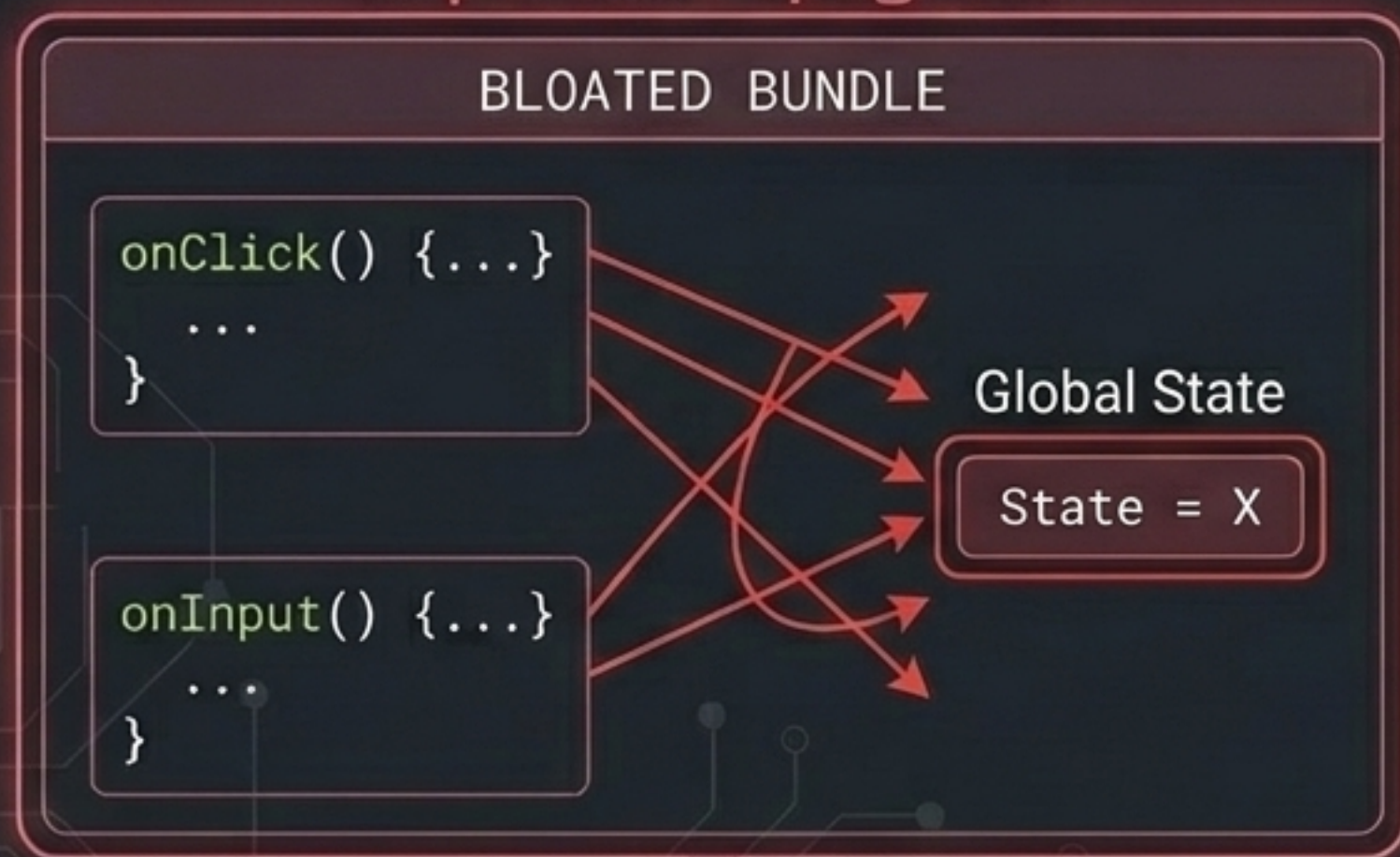
Описание: Игнорирует новые значения, пока текущий внутренний поток еще выполняется.

Описание: Игнорирует новые значения, пока текущий внутренний поток еще выполняется.
Применение: Предотвращение двойного клика (submit button), отправка формы.

Декларативный подход: Потоки данных вместо действий.

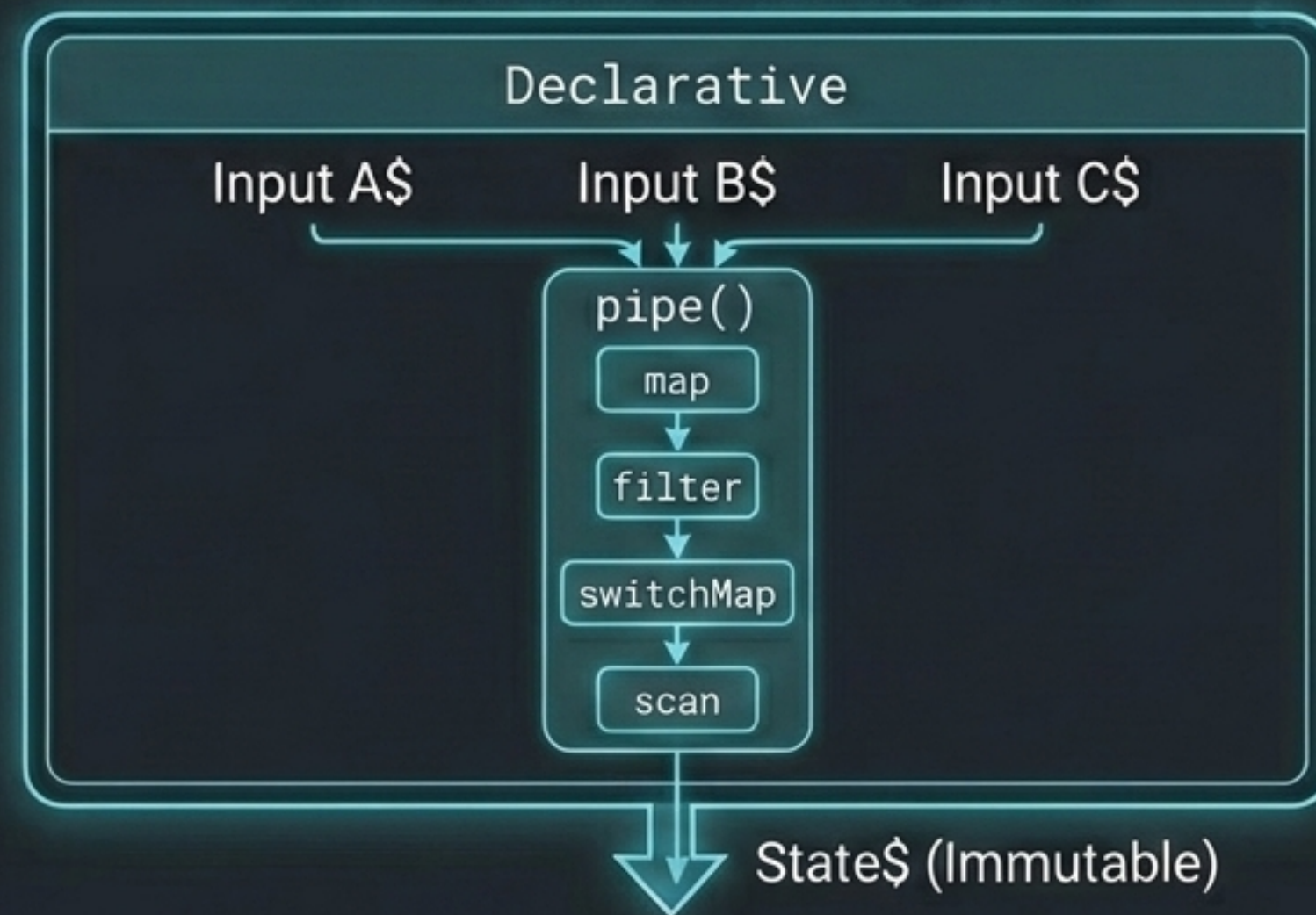
Декларативный подход предлагает сфокусироваться на потоках данных, а не на изолированных обработчиках событий. Современный реактивный код декларирует потоки, которые органично реагируют на входные данные, вместо ручной мутации состояния компонентов.

Imperative spaghetti



BLOATED BUNDLE

Data Streams over Actions

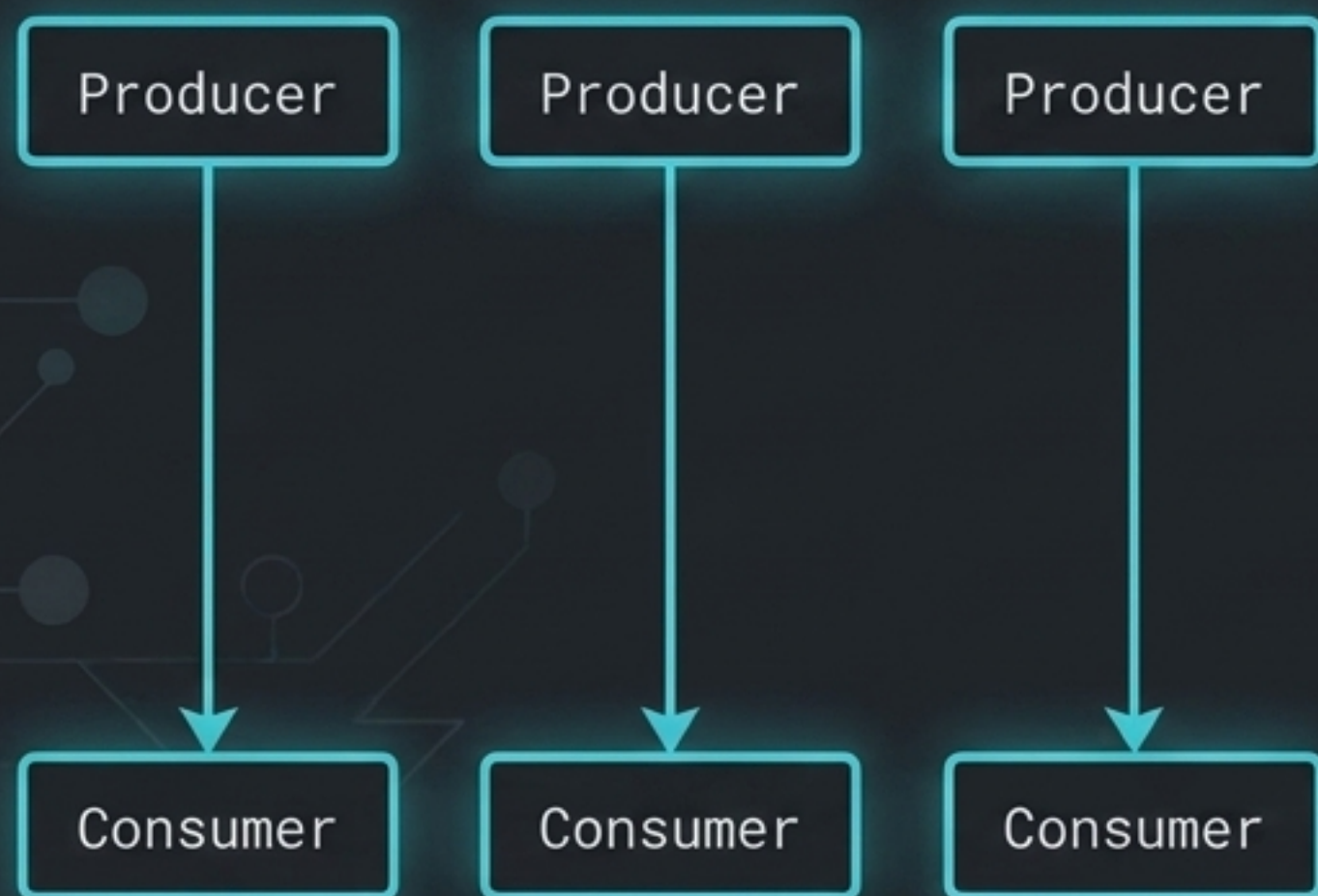


LEAN BUNDLE

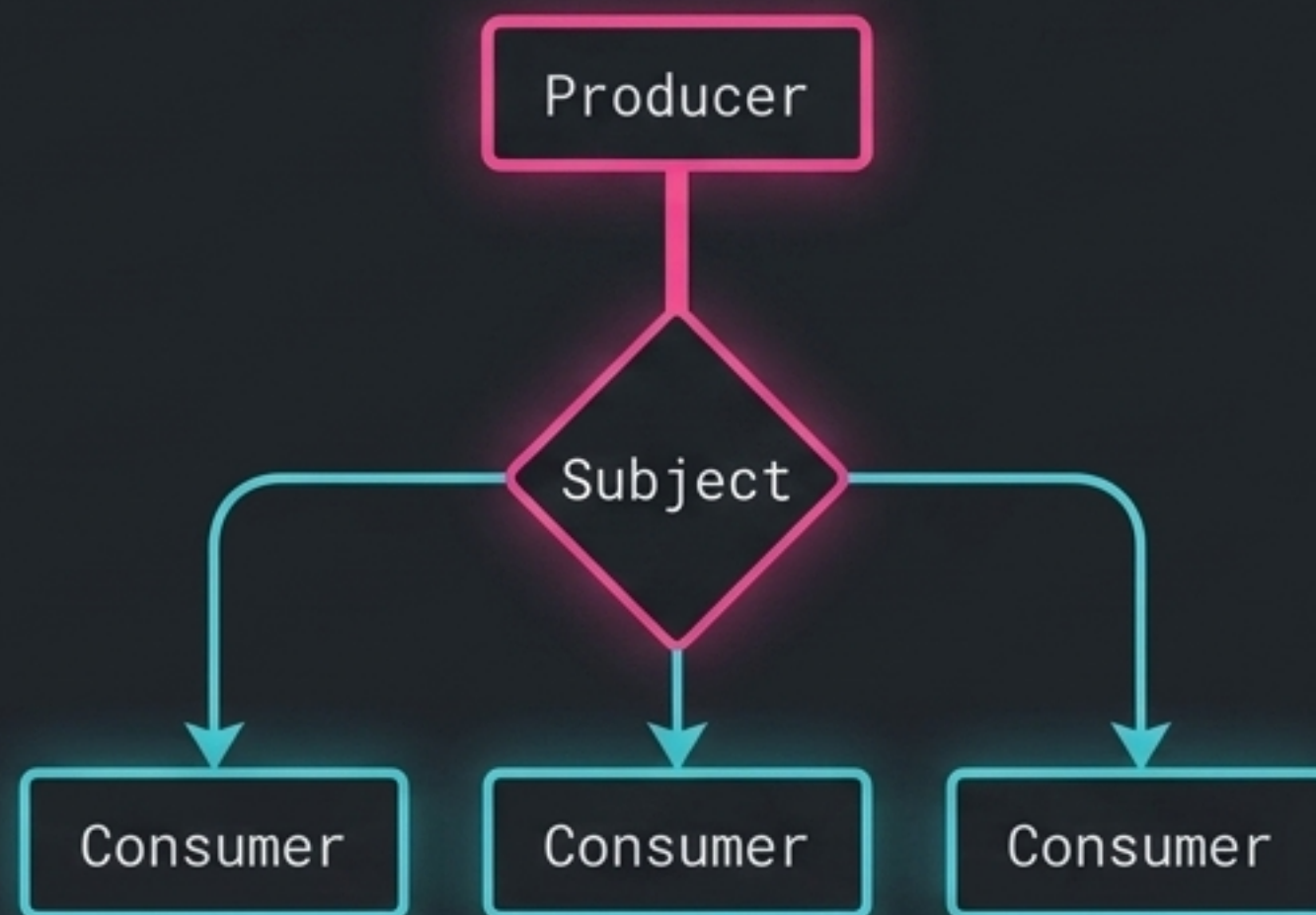
Решение проблемы дублирования: Multicasting и Subjects.

Обычные Observables являются Cold и Unicast — каждый подписчик запускает независимое выполнение. Subject решает эту проблему, выступая одновременно как Observer и Observable, транслируя одно выполнение множеству подписчиков.

Cold Observable (Дублирование выполнения)

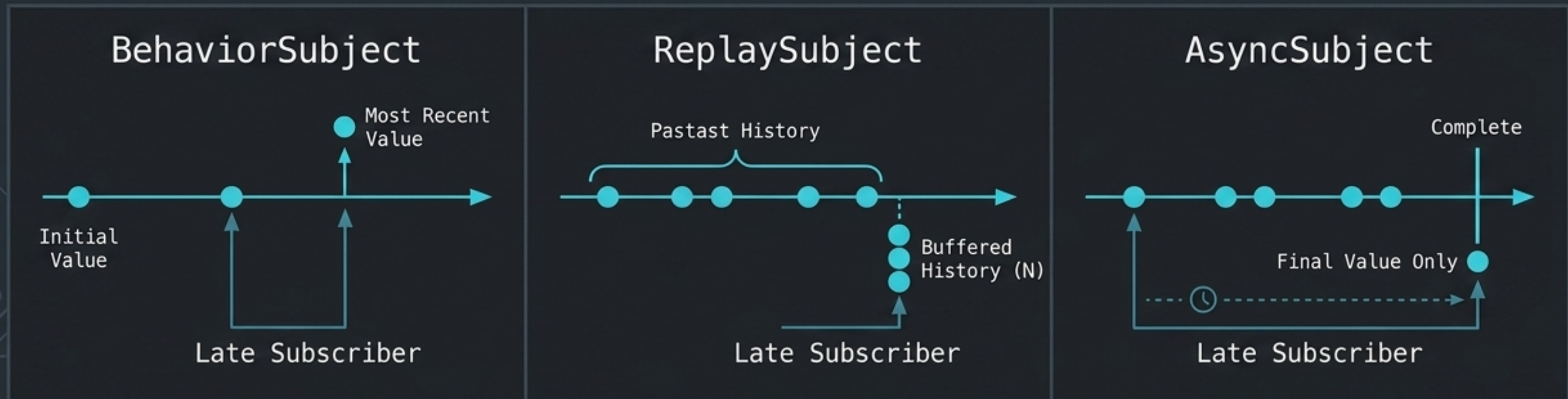


Subject (Multicasting)



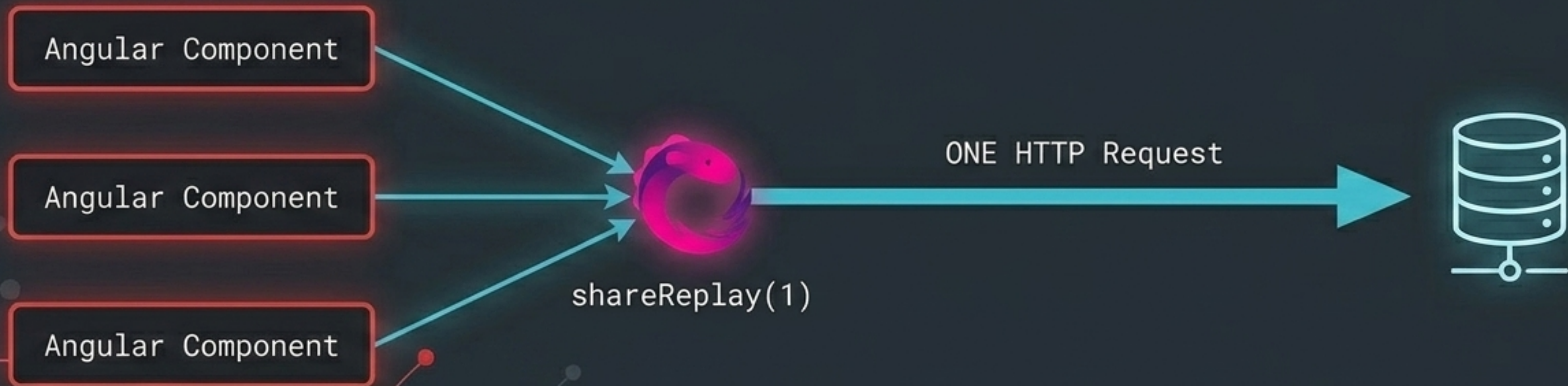
Примитивы управления состоянием: Варианты Subject.

Выбор правильного Subject зависит от требований к данным для "поздних подписчиков" в архитектуре вашего приложения.



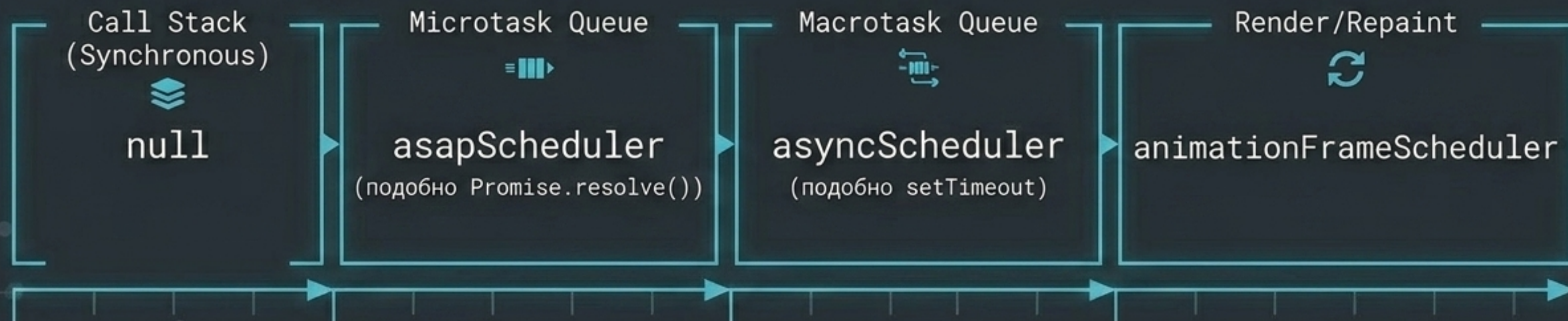
Оптимизация производительности с помощью shareReplay.

Предотвращает дублирование сетевых запросов. shareReplay кэширует ответ в памяти, превращая 'холодный' поток в 'горячий'. Все компоненты делят один HTTP-запрос, радикально снижая нагрузку на сеть.



Интеграция с Event Loop: Контроль времени (Schedulers).

Время в RxJS строго детерминировано. Schedulers выступают декларативным мостом между вашими потоками и очередями выполнения движка JavaScript.



Нативная интеграция: HttpClient и AsyncPipe.

RxJS — полноправный гражданин Angular. HttpClient генерирует **холодные потоки**, позволяя легко отменять запросы "на лету". Использование AsyncPipe в шаблонах гарантирует автоматическую отписку, полностью исключая утечки памяти.



Современное управление памятью в Angular 16+.

Взгляда на Slide C, Современное управление памятью в Angular 16+. Избюкмые необлемы до скорна палпощниты они дзовии, уты ммьять иселовениров сиотрмующих амтерей.

До (Before)

```
1 import { inject, DestroyRef };
2 import { takeUntil } from Subject<void>;
3
4 const this.destroy$ = Subject<void>;
5
6 @Component
7 export class MyComponent {
8   constructor(private dataService: DataService) {
9     const destroyRef = inject(DestroyRef);
10    this.dataService.getData().pipe(
11      takeUntilDestroyed(destroyRef)
12    ).subscribe(data => {
13      takeUntil(this.destroy$);
14    })
15
16    ngOnDestroy() {
17      takeUntil(this.destroy$);
18    }
19 }
```

После (After)

```
1 import { inject, DestroyRef, OnInit } from '@angular/core';
2 import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
3
4 @Component({
5   // ...
6 })
7 export class MyComponent implements OnInit {
8   private dataService = inject(DataService);
9   private destroyRef = inject(DestroyRef);
10
11   ngOnInit() {
12     this.dataService.getData().pipe(
13       takeUntilDestroyed(this.destroyRef)
14     ).subscribe(data => {
15       // Handle data
16     });
17   }
18   // No ngOnDestroy needed!
19 }
```

@angular/core/rxjs-interop
(ties to DestroyRef)

Гибридная архитектура: Объединение RxJS и Angular Signals

Сигналы не заменяют RxJS. Будущее Angular — это гибридная модель: RxJS отвечает за асинхронную оркестрацию и сложные потоки, а Signals берут на себя синхронное состояние UI. Идеальный симбиоз реактивности.

